

Using *sabinaHSBM* for link prediction and network reconstruction with “marginal_all” method

This guide shows a simple use case based on the “marginal_all” method in *sabinaHSBM* to detect both missing (false negatives) and/or spurious (false positives) links in an incomplete and/or error-prone network.

We use the example dataset **dat2** included in the package to show key functionalities, including data preparation, link prediction, and network reconstruction.

Requirements and installation

There are **two ways** to use the package:

Option A: Docker (recommended)

Non-native UNIX users or those who prefer to avoid manual dependency installation can use the provided **ready-to-use Docker image** (includes R, Python, all dependencies, and *sabinaHSBM* pre-installed (see Supporting Information S1 for Docker container setup).

Once the container is running, simply load the package in your R session with:

```
library(sabinaHSBM)
```

Option B: Native UNIX installation

Native UNIX users can run *sabinaHSBM* locally **if** their system includes: - R (version $\leq 4.0.4$) with all required R packages (listed below) - Python $\leq$ with the **graph-tool** library (version ≤ 2.45)

```
install.packages("remotes")
# If the package is not installed, install sabinaHSBM from GitHub
if (!requireNamespace("sabinaHSBM", quietly = TRUE)) {
  remotes::install_github("h-lima/sabinaHSBM")
}

# Load sabinaHSBM
library(sabinaHSBM)
```

Install required R packages if not already available:

```
list.of.packages <- c(
  "dplyr",
  "reshape2",
  "reticulate",
  "stringr",
  "tidyr",
  "ROCR"
)

new.packages <- list.of.packages[!(list.of.packages %in% installed.packages()[, "Package"])]
if (length(new.packages) > 0) {
```

```

install.packages(new.packages, dependencies = TRUE)
}

for (package in list.of.packages) {
  library(package, character.only = TRUE)
}

# Record starting time (optional)
start_time <- Sys.time()

```

Load example data

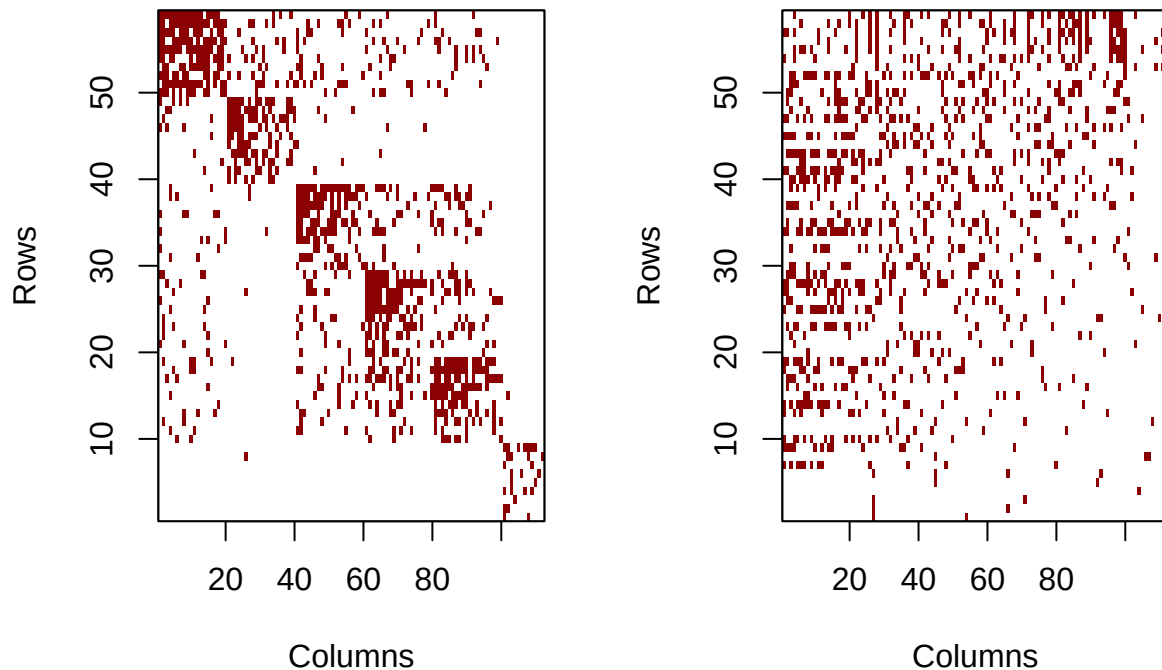
The dataset `dat2` is a binary matrix representing a hypothetical species interactions network. Columns and rows correspond to two different types of nodes (e.g., hosts and parasites), and links (values of 1, in red) represent interactions between them while 0 (in black) represent lack of an observed interaction. These links are distributed in a structured pattern, with no missing information, to **focus the example on identifying spurious links**.

```

# Load the dataset
#data(dat2, package = "sabinaHSEB")
#dat <- dat2
load("../testing/dat2.RData")

# Plot the original matrix
par(mfrow = c(1, 2))
plot_interaction_matrix(adj_mat = dat, order_mat = FALSE)
plot_interaction_matrix(adj_mat = dat, order_mat = TRUE)

```



To simulate spurious links and address the `marginal_all` method, we randomly add a small number of false positives (1) to the matrix. This controlled modification allows us to demonstrate how the method can work with both spurious links (observed but potentially erroneous interactions, false positives) and missing links (unobserved but likely interactions, false negatives).

```
# Add spurious links artificially
set.seed(123)

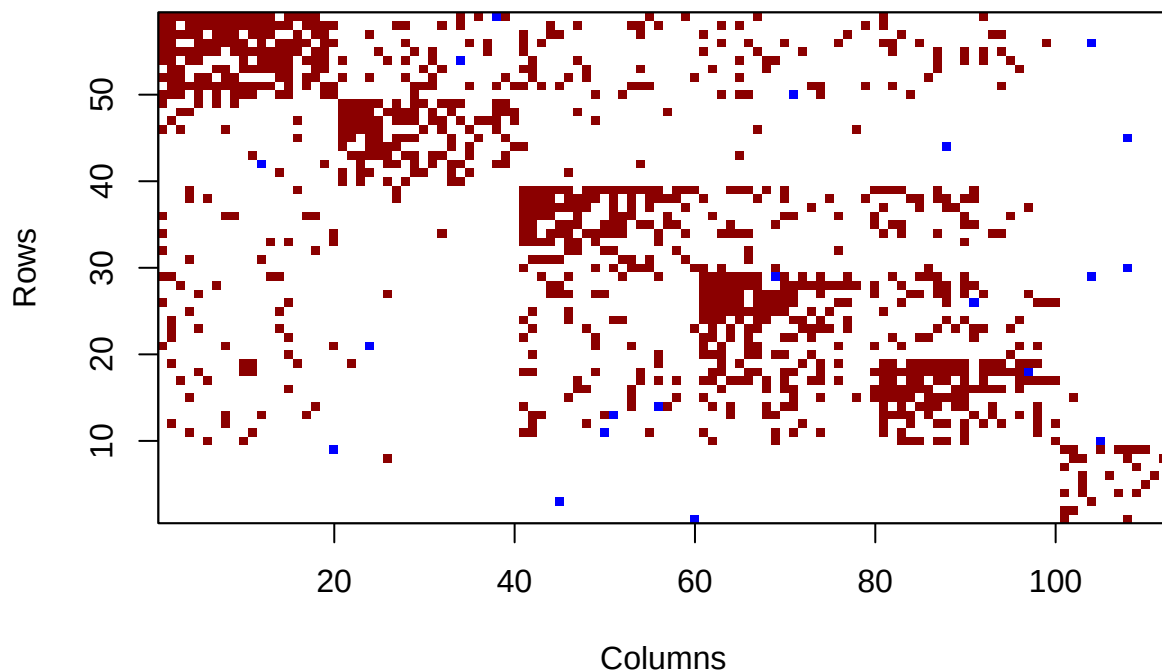
n_spurious <- ceiling(sum(dat) * 0.02)

# Find some random spurious
zero_indices <- which(dat == 0)
sampled_indices <- sample(zero_indices, n_spurious)
spurious_links <- arrayInd(sampled_indices, dim(dat))

total_spurious <- nrow(spurious_links)

dat[sampled_indices] <- 1

# Plot the matrix with spurious
dat_spur <- dat
dat_spur[sampled_indices] <- 2
plot_interaction_matrix(dat_spur,
  col = c("white", "red4", "blue1"),
  order_mat = FALSE)
```



Prepare input data for HSBM

The `hsbm.input` function cleans the dataset and creates cross-validation folds by temporarily holding out subsets of observed links. Internally, it ensures that each fold masks a representative sample. In our example, it randomly splits all observed links into five similar-sized subsets (because here we use 5-fold cross-validation), holding out one subset per fold to test the model’s ability to recover missing links. We set “`max_held_per_fold`” to the number of spurious to keep the data more balanced, otherwise, number of removed links would surpass `total_spurious` by a large margin hindering inference, since to detect spurious it is better to have the dataset as complete as possible.

```
# Prepare input data
n_folds <- 3      # Number of folds for cross-validation

myInput <- hsbm.input(
  dat,            # Binary matrix of observed links
  n_folds = n_folds,
  max_held_per_fold = total_spurious
)
```

For the purposes of this example, the spurious links artificially added are labeled with the “`spurious_edge`” tag using the code below. This labeling allows for tracking these links throughout the analysis and network reconstruction process, although in typical use, such labeling would not be necessary.

```
for(i in 1:length(myInput$edgelist)){
  el <- myInput$edgelist[[i]]
  # Turn node ids into row and col indices
```

```

rows <- as.numeric(el[, 1]) + 1
cols <- as.numeric(el[, 2]) - nrow(myInput$data) + 1
rows_cols <- paste0(rows, "_", cols)
# Get spurious links indices
spurious_rows_cols <- paste0(spurious_links[, 1], "_", spurious_links[, 2])
spurious_loc <- which(rows_cols %in% spurious_rows_cols)
# Add spurious edge type to edge lists
el[spurious_loc, ]$x <- 1
el[spurious_loc, ]$edge_type <- "spurious_edge"
myInput$edgelist[[i]] <- el
}

```

```

# Summarizes network characteristics
summary(myInput)

```

```

##   n_rows n_cols n_links rows_single_link cols_single_link possible_links
## 1      59    112    1009              0              3          6608

```

Predict link probabilities with “marginal_all”

The `hsbm.predict` function applies the HSBM to predict marginal posterior probabilities of all links—both observed (1s) and unobserved (0s)—so you can identify spurious and missing links in one step. The function works directly with the processed input created by `hsbm.input`. Computation can be intensive, but parallelization across cores is supported.

```

# Generate HSBM predictions
myPred <- hsbm.predict(
  myInput,           # Input data processed by hsbm.input()
  iter = 10000,      # Number of iterations
  wait = 10000,      # Number of iterations for MCMC equilibration
  rnd_seed = 123,    # Sets seed in python environment for reproducibility
  method = "marginal_all", # Method for link prediction
  save_blocks = TRUE, # Save group assignments
  save_pickle = FALSE, # Save results as pickle files,
  save_plots = FALSE, # Save hierarchical edge bundling plots
  n_cores = 3
)

```

Predicted link probabilities and group assignments are stored for each fold.

Below, we extract the link probabilities (p) for fold 1. For example, a high probability ($p = 0.91$) of an undocumented link between host 1 and parasite 2 would suggest that this interaction is likely real but was missed due to sampling limitations. Conversely, it may assign a low p (e.g., 0.12) to a recorded interaction between host and parasite, indicating it might be spurious.

```

# View probabilities for fold 1
probabilities_fold1 <- myPred$probs[[1]]
head(probabilities_fold1)

```

```

##   v1 v2 p v1_names v2_names edge_type
## 1  0 59 1    row1    col1 documented
## 2  0 60 1    row1    col2 documented
## 3  0 61 1    row1    col3 documented
## 4  0 97 1    row1   col39 documented
## 5  0 64 1    row1    col6 documented
## 6  0 94 1    row1   col36 documented

```

The group assignments provide the hierarchical clustering structure of nodes for each fold. Let's extract and examine the group assignments for fold 1:

```
# View the group/block assignments for fold 1
groups_fold1 <- myPred$groups[[1]]

# Filter one type of nodes (nodes in columns)
vnames <- colnames(myPred$data)
groups_fold1 <- groups_fold1 %>% filter(name %in% vnames)
g_cols <- grep("^G", names(groups_fold1))
groups_fold1[g_cols] <- lapply(groups_fold1[g_cols], sort)

#print(head(groups_fold1))
print(lapply(myPred$groups, function(x) apply(x, 2, function(y) length(unique(y)))))

## [[1]]
## node   G1   G2   G3   G4   G5 name
##  171   11    3    2    1    1  171
##
## [[2]]
## node   G1   G2   G3   G4   G5 name
##  171    9    3    2    1    1  171
##
## [[3]]
## node   G1   G2   G3   G4   G5 name
##  171   11    3    2    1    1  171
```

Hierarchical group assignments provide insight into how nodes (e.g., hosts) are organized across multiple levels. At the first level (G1) nodes are divided into specific groups, reflecting fine-scale patterns. Moving to higher levels (G2, G3, G4) these groups (or communities) are progressively aggregated, revealing broader patterns and relationships or communities.

Next, we will explore how the model assigns specific probabilities p to different types of links (“documented,” “held_out,” and “spurious_edge”) for each fold to evaluate HSBM ability to correctly identify them. Documented links, which are real observations (true positives), should have probabilities close to 1. Held-out links, which are real observations but transformed to 0s during training for validation, should also show high probabilities. In contrast, spurious links, artificially generated (initially unobserved links transformed to 1s), should have probabilities close to 0 (unless it could be a missing link), as the model should recognize them as false links (false negatives).

```
fold_res <- list()
count_higher <- vector(length = n_folds)
for(i in 1:n_folds){
  doc <- dplyr::filter(myPred$probs[[i]], edge_type == "documented")
  undoc <- dplyr::filter(myPred$probs[[i]], edge_type == "reconstructed")
  held <- dplyr::filter(myPred$probs[[i]], edge_type == "held_out")
  spur <- dplyr::filter(myPred$probs[[i]], edge_type == "spurious_edge")
  fold_data <- c(i,
                 nrow(doc), sum(doc$p < 0.95),
                 nrow(spur), sum(spur$p < 0.95),
                 nrow(undoc), sum(undoc$p > 0.05),
                 nrow(held), sum(held$p > 0.05))
  fold_res[[i]] <- fold_data

  # Count held links with p higher than all undoc
  for(held_link in held$p){
```

```

    if(all(undoc$p < held_link)){
      count_higher[i] <- count_higher[i] + 1
    }
  }
}

res_summary <- do.call(rbind, fold_res)
colnames(res_summary) <- c("Fold",
  "Nr doc", "Doc p < 0.95",
  "Nr spur", "Spur p < 0.95",
  "Nr undoc", "Undoc p > 0.05",
  "Nr held", "Held p > 0.05")
# We use kable function from kable package to prettify output here
kable(res_summary)

```

Fold	Nr doc	Doc p < 0.95	Nr spur	Spur p < 0.95	Nr undoc	Undoc p > 0.05	Nr held	Held p > 0.05
1	970	4	20	3	4816	457	19	10
2	969	13	20	5	4592	686	20	14
3	969	8	20	4	4646	431	20	9

Since we use “marginal_all” method, all table numbers only take into account the links sampled during MCMC iterations.

Network Reconstruction

The `hsbm.reconstructed` function generates a reconstructed binary interaction matrix by combining predictions from all folds. The predicted matrix is transformed to binary values using a user-specified threshold.

```

# Network reconstruction
myReconst <- hsbm.reconstructed(
  myPred,                # Predictions processed by hsbm.predict
  rm_documented=FALSE,   # Remove documented links during validation
  na_treatment = "ignore_na", # Handle NA values in predictions
  threshold = 0.5,        # Binarization threshold
  consistency_matrix = "average_thresholded" # Combine fold predictions
)

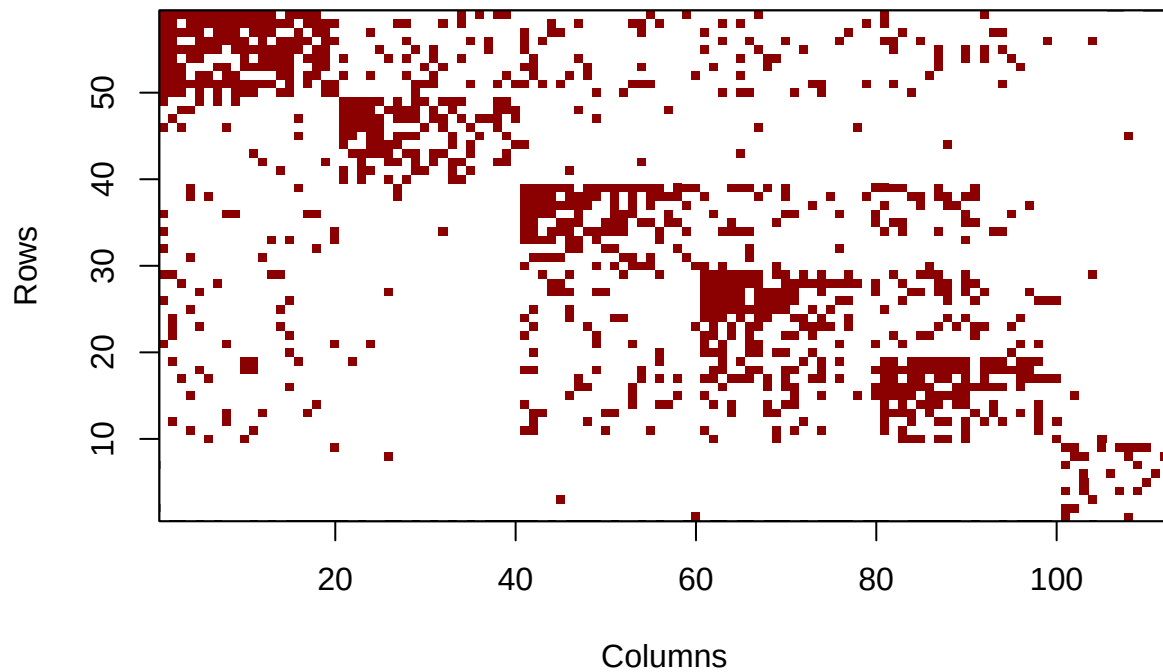
```

This output includes the final averaged probability matrix, the final reconstructed binary matrix, and evaluation metrics. These results highlight the predicted interactions, showcasing the method’s ability to detect spurious and missing links effectively. Let’s explore some of the key outputs.

```

# Visualize the reconstructed binary matrix
plot_interaction_matrix(myReconst$new_mat, order_mat = FALSE)

```



```
print(myReconst$new_mat[sampled_indices])
```

```
## [1] 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1
```

```
# Plot the matrix with spurious
```

```
cols <- c("white", "red4")
```

```
dat_spur <- myReconst$data
```

```
new_spur <- which(myReconst$data == 1 & myReconst$new_mat == 0)
```

```
new_missing <- which(myReconst$data == 0 & myReconst$new_mat == 1)
```

```
if(length(new_spur) != 0){
```

```
  dat_spur[new_spur] <- 2
```

```
  cols <- c(cols, "blue4")
```

```
}
```

```
if(length(new_missing) != 0){
```

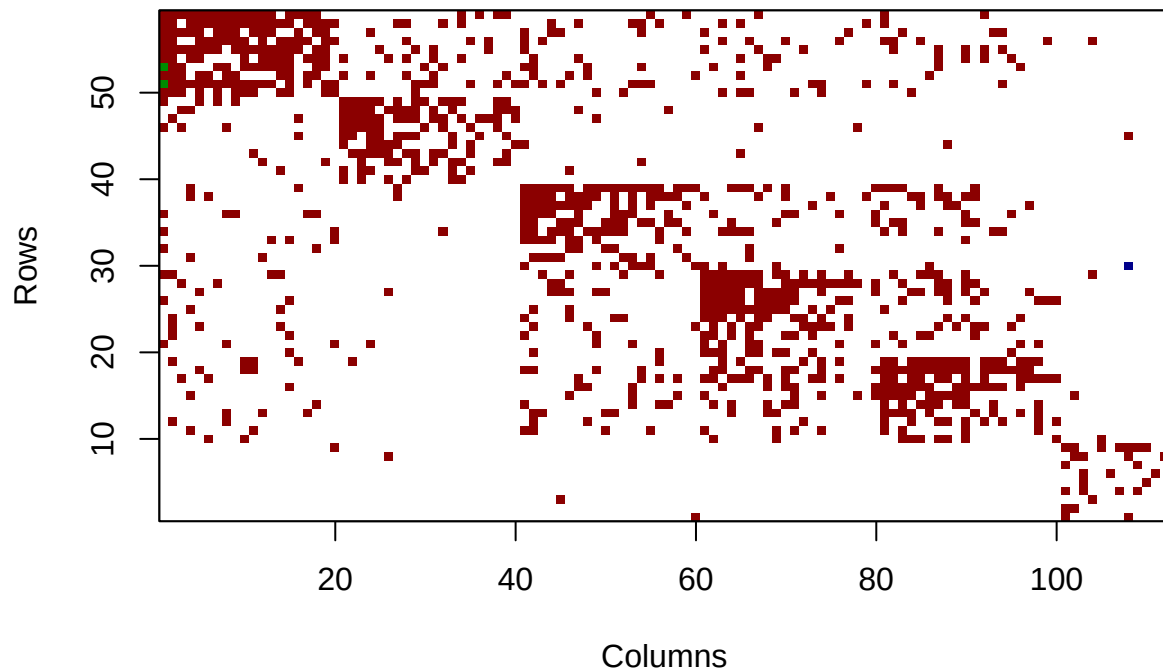
```
  dat_spur[new_missing] <- 3
```

```
  cols <- c(cols, "green4")
```

```
}
```

```
plot_interaction_matrix(dat_spur,
```

```
  col = cols, order_mat = FALSE)
```

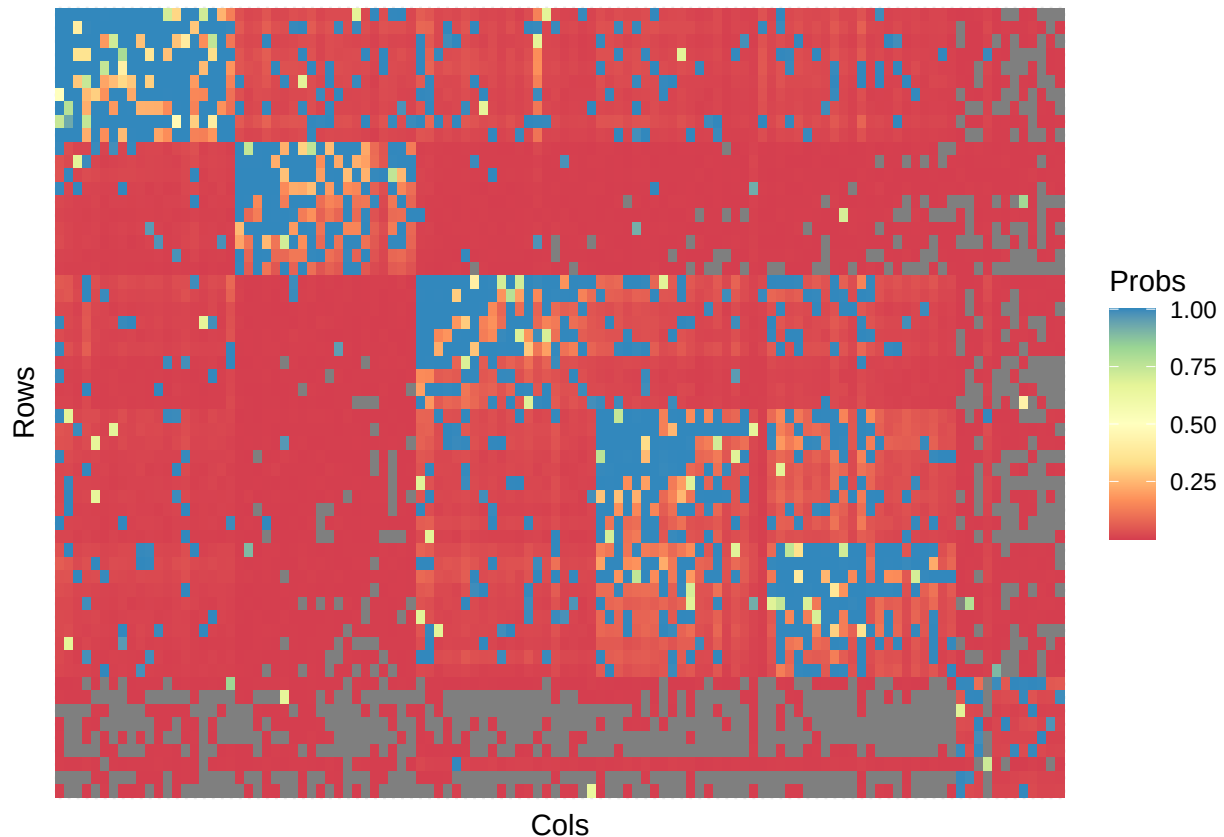
```
library(ggplot2)
library(RColorBrewer)

mat_avg <- apply(simplify2array(myReconst$pred_mats), 1:2, mean,
                 na.rm = TRUE)
print(mat_avg[sampled_indices])

## [1] 0.9908658 1.0000000 1.0000000 0.9716972 0.7009034 0.6366970 0.9994666
## [8] 1.0000000 0.9995666 0.8121145 0.4252759 0.9991999 1.0000000 1.0000000
## [15] 0.9963996 0.9934993 0.9944328 0.8927559 0.8233157 1.0000000

melted_matrix <- reshape2::melt(mat_avg)

row_names <- rownames(mat_avg)
colnames(melted_matrix) <- c("Rows", "Cols", "Probs")
ggplot(melted_matrix, aes(Cols, Rows, fill= Probs)) +
  geom_tile() +
  scale_y_discrete(limits = rev(row_names)) +
  scale_fill_distiller(palette = "Spectral", direction = 1) +
  theme_minimal() +
  theme(axis.text = element_blank())
```



```
# View the reconstruction summary and evaluation metrics
summary(myReconst)
```

```
## $`Reconstructed Network Metrics`
##   obs_links unobs_links pred_links kept_links spurious_links missing_links
## 1      1009       5599      1010       1008             1             2
##
## $`Evaluation Metrics`
##   mean_RLF mean_auc mean_aucpr mean_yPRC mean_prec mean_sens mean_spec
## 1 0.03333333 0.9970955 0.9938692 0.1526937 0.9976407 0.9781962 0.9995833
##   mean_ACC mean_ERR mean_tss
## 1 0.9963176 0.003682405 0.9777795
```

The summary provides the number of spurious and missing links, as well as key evaluation metrics, such as the recovery link ratio (RLF).

Let's now examine the top 10 predicted links that are most likely to be spurious (false positives) by visualizing the probabilities of "documented" links.

```
# Visualize the top most likely spurious links
top_links_spurious <- top_links(myReconst,
                                n = 10,
                                edge_type = "documented")
print(top_links_spurious)
```

```
##   v1_names v2_names      p      sd
## 1   row30   col108 0.4252759 0.3780782
## 2   row59   col160 0.6366970 0.4669748
```

```
## 3    row37    col15 0.6532987 0.4830695
## 4    row52    col26 0.6600993 0.2974482
## 5    row33    col15 0.6620995 0.5007396
## 6     row8    col48 0.6621662 0.5044444
## 7    row24    col17 0.6628329 0.1904176
## 8    row32    col78 0.6635330 0.5072563
## 9    row48    col12 0.6683002 0.1898266
## 10   row3     col54 0.6697336 0.1885793
```

This example was tailored to find spurious links and the model didn't identify any missing links at the threshold of 0.5, which is a very conservative threshold. However we note that on average, 2% of the held-out links had a higher probability than all undocumented links across folds.

We show how we can rank the most likely missing links according to the model.

```
# Visualize the top most likely spurious links
top_links_missing <- top_links(myReconst,
                               n = 10,
                               edge_type = "undocumented")
print(top_links_missing)
```

```
##    v1_names v2_names      p      sd
## 1    row9    col1 0.7374404 0.015960722
## 2    row7    col1 0.5032503 0.016697877
## 3    row9   col14 0.4709804 0.017798669
## 4    row5    col8 0.4179751 0.020331441
## 5   row21   col47 0.4131413 0.050946859
## 6    row2    col3 0.4065407 0.062262899
## 7    row9   col18 0.4041071 0.001714598
## 8    row3   col15 0.4009068 0.021872952
## 9   row43   col83 0.3952062 0.123075892
## 10   row44   col87 0.3733707 0.119006678
```

If the task is to do network reconstruction by inserting the most plausible missing links, we can use the probabilities from the `marginal_all` method as 'scores' in a supervised binary classification, which is implemented only for missing links. This is achieved by following the steps in Supporting Information S1, but setting "method = 'marginal_all'" in `hsbm.predict()` and optionally "rm_documented = 'FALSE'" in `hsbm.reconstructed()`.

This document demonstrates the use of the *sabinaHSBM* package for network reconstruction. By applying the "marginal_all" method, we showcased how likely spurious and missing links can be identified and addressed in complex networks.

Testing...

```
# Network reconstruction
myReconst <- hsbm.reconstructed(
  myPred,                # Predictions processed by hsbm.predict
  rm_documented = FALSE, # Use of documented entries during validation
)
print(myReconst$stats)
```

```
##    folds      auc      aucpr      yPRC      thresh n_heldout pred_held_ones n_ones
## 1     1 0.9975317 0.9944367 0.1526937 0.7728773      20          0.1    1009
## 2     2 0.9968752 0.9940255 0.1526937 0.4699470      20          0.0    1009
## 3     3 0.9968796 0.9931454 0.1526937 0.4363436      20          0.0    1009
```

```
##   pred_tot_ones total_pred_ones precision      sens      spec      ACC
## 1      0.9811695              990 1.0000000 0.9811695 1.0000000 0.9971247
## 2      0.9762141              989 0.9959555 0.9762141 0.9992856 0.9957627
## 3      0.9791873              992 0.9959677 0.9791873 0.9992856 0.9962167
##           ERR           tss
## 1 0.002875303 0.9811695
## 2 0.004237288 0.9754997
## 3 0.003783293 0.9784729
```

Computing characteristics

```
# Show processing time and computer characteristics (optional)
end_time <- Sys.time()
cat("The processing time of this script took: ",
    round(as.numeric(difftime(end_time, start_time, units = "mins", 1))), "minutes\n")
```

```
## The processing time of this script took: 19 minutes
```

This analysis was performed on a Dell Inspiron notebook with the following characteristics:

- Processor (CPU): Intel Core i7-7500U @ 3.5GHz
- Memory (RAM): 16 GB
- Operating System: Ubuntu 24.04 (run on a docker image)
- R Version: 4.3.3